Solution: Fibonacci numbers

Problem Statement

Write a function to calculate the nth Fibonacci number.

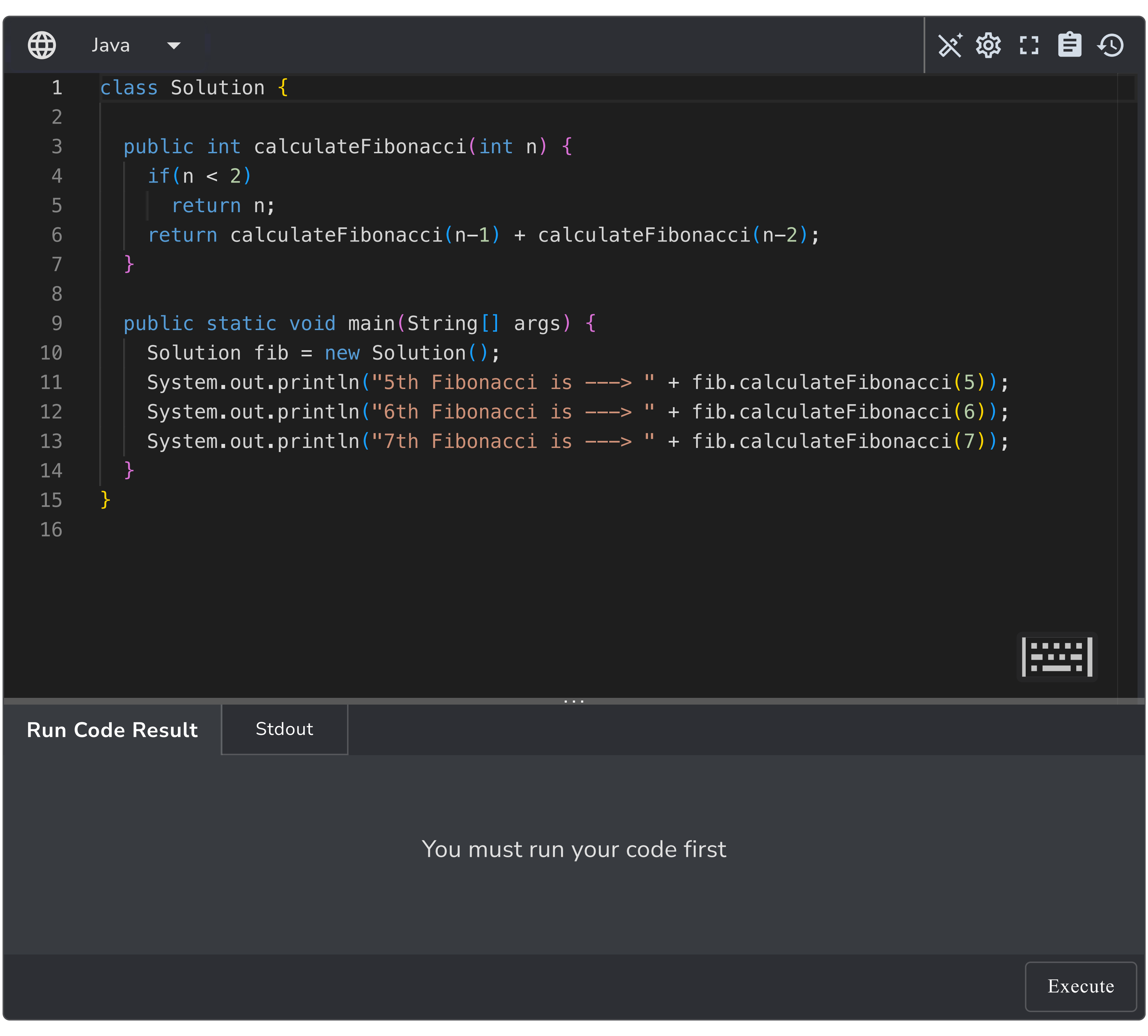
Fibonacci numbers are a series of numbers in which each number is the sum of the two preceding numbers. First few Fibonacci numbers are: 0, 1, 1, 2, 3, 5, 8, …

Mathematically we can define the Fibonacci numbers as:

```
Fib(n) = Fib(n-1) + Fib(n-2), for n > 1

Given that: Fib(0) = 0, and Fib(1) = 1
```

Basic Solution

A basic solution could be to have a recursive implementation of the mathematical formula discussed above:

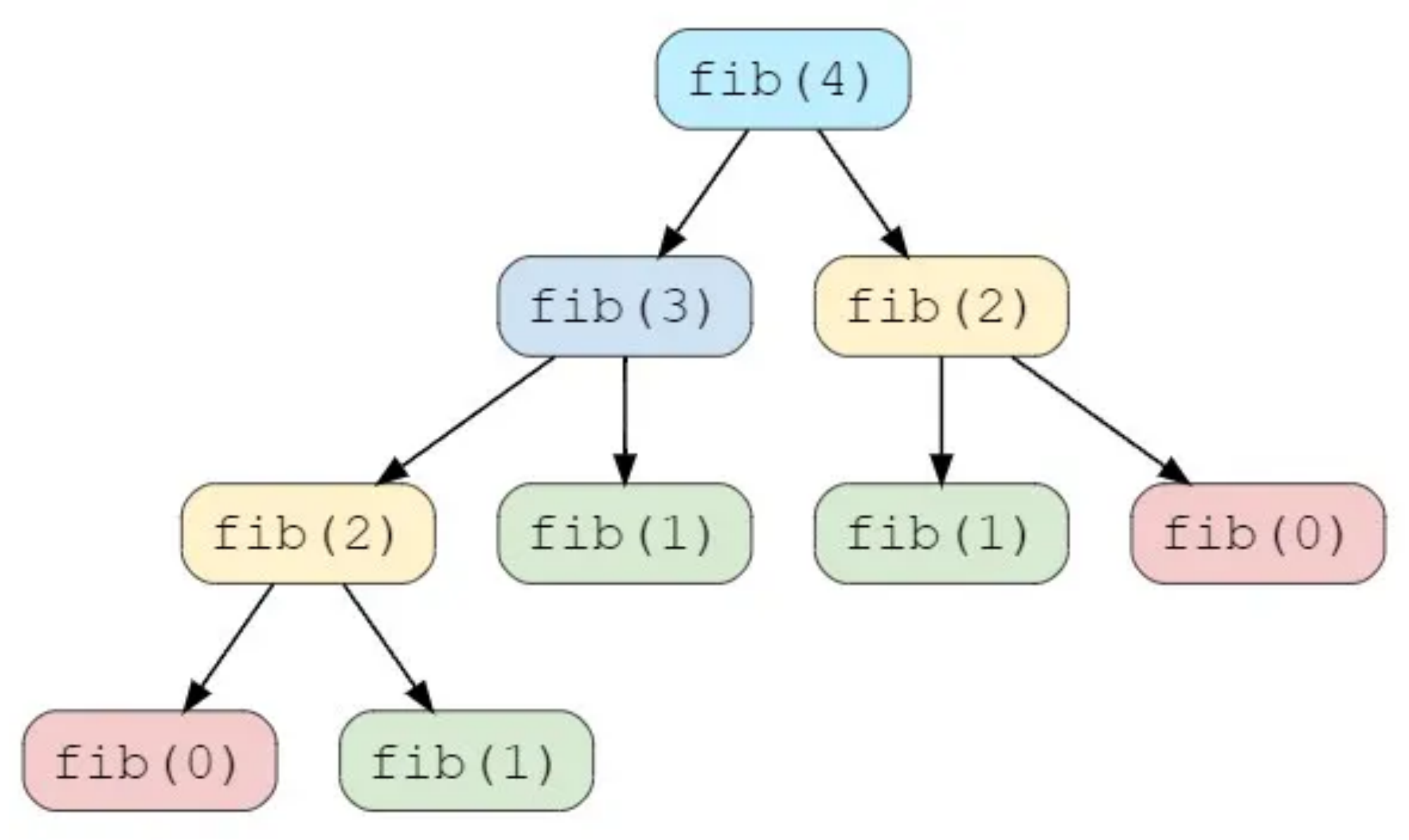


```java
class Solution {

  public int calculateFibonacci(int n) {
    if(n < 2)
      return n;
    return calculateFibonacci(n-1) + calculateFibonacci(n-2);
  }

  public static void main(String[] args) {
    Solution fib = new Solution();
    System.out.println("5th Fibonacci is ---> " + fib.calculateFibonacci(5));
    System.out.println("6th Fibonacci is ---> " + fib.calculateFibonacci(6));
    System.out.println("7th Fibonacci is ---> " + fib.calculateFibonacci(7));
  }
}
```

You must run your code first

The time complexity of the above algorithm is exponential $O(2^n)$ as we are making two recursive calls in the same function. The space complexity is $O(n)$ which is used to store the recursion stack.

Let's visually draw the recursion for `CalculateFibonacci(4)` to see the overlapping subproblems:

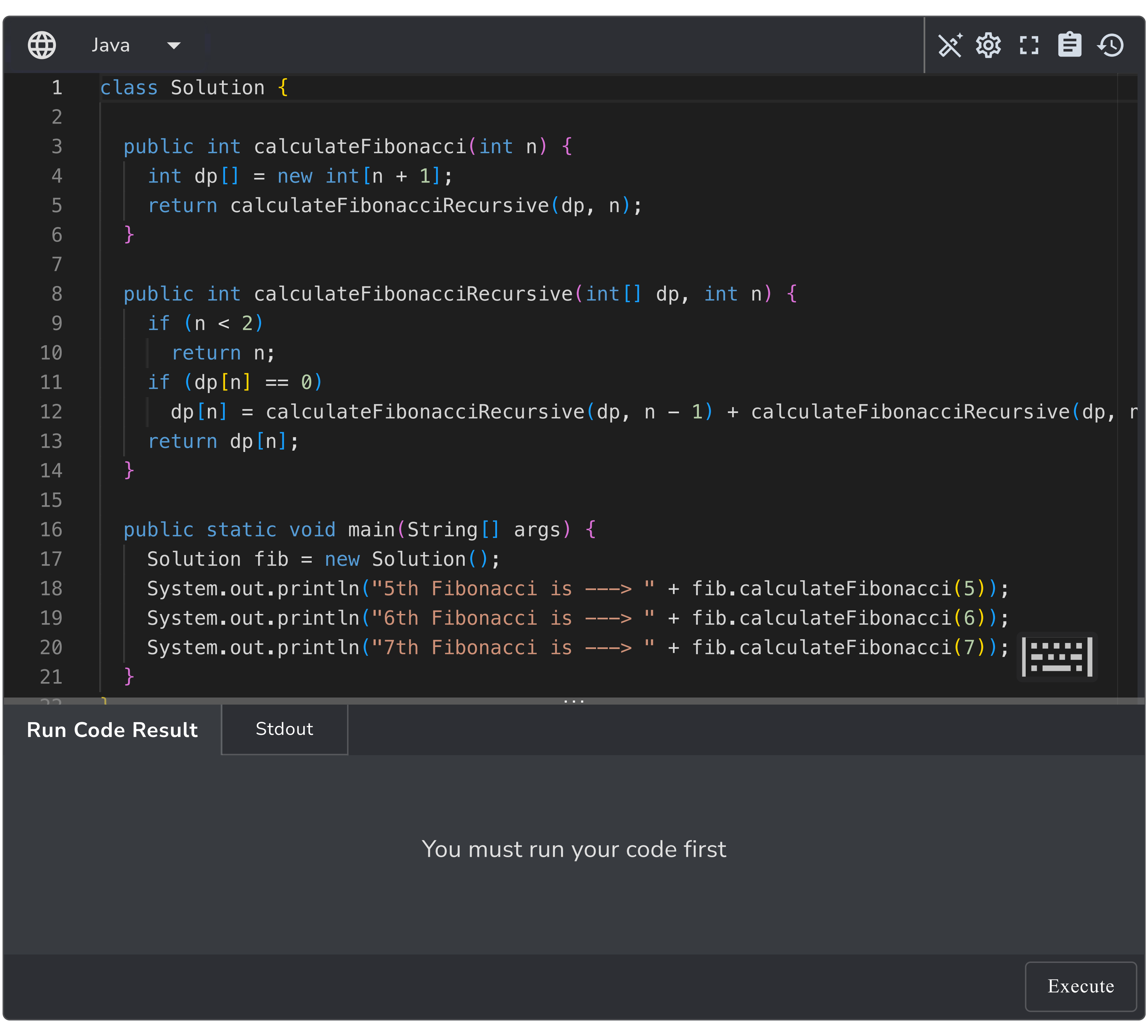


Recursion tree for calculating Fibonacci numbers

We can clearly see the overlapping subproblem pattern: `fib(2)` has been called twice and `fib(1)` has been called thrice. We can optimize this using memoization to store the results for subproblems.

Top-down Dynamic Programming with Memoization

We can use an array to store the already solved subproblems. Here is the code:



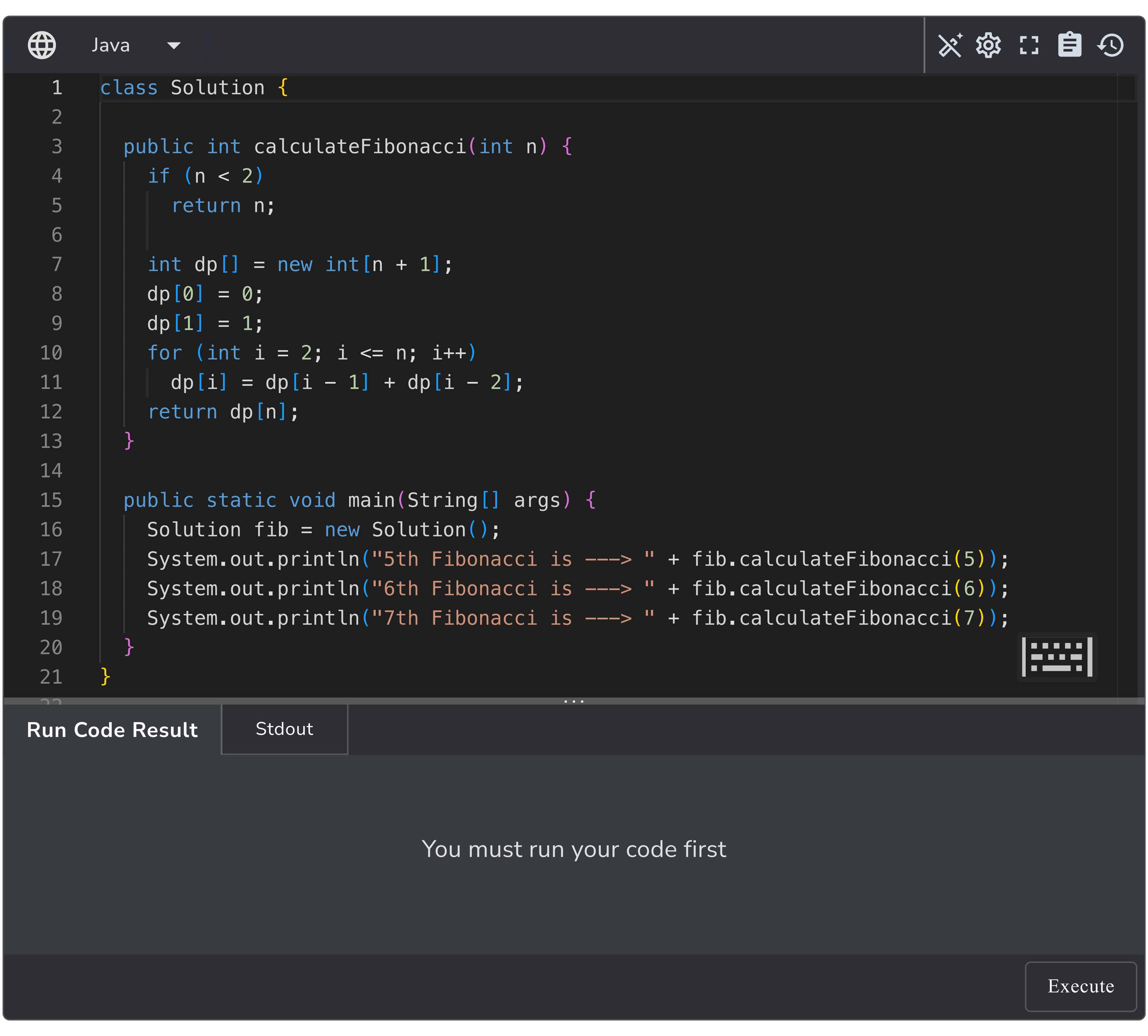
```java
class Solution {

  public int calculateFibonacci(int n) {
    int dp[] = new int[n + 1];
    return calculateFibonacciRecursive(dp, n);
  }

  public int calculateFibonacciRecursive(int[] dp, int n) {
    if (n < 2)
      return n;
    if (dp[n] == 0)
      dp[n] = calculateFibonacciRecursive(dp, n - 1) + calculateFibonacciRecursive(dp, 
    return dp[n];
  }

  public static void main(String[] args) {
    Solution fib = new Solution();
    System.out.println("5th Fibonacci is ---> " + fib.calculateFibonacci(5));
    System.out.println("6th Fibonacci is ---> " + fib.calculateFibonacci(6));
    System.out.println("7th Fibonacci is ---> " + fib.calculateFibonacci(7));
  }
```

You must run your code first

Bottom-up Dynamic Programming

Let's try to populate our `dp[]` array from the above solution, working in a bottom-up fashion. Since every Fibonacci number is the sum of the previous two numbers, we can use this fact to populate our array.

Here is the code for the bottom-up dynamic programming approach:



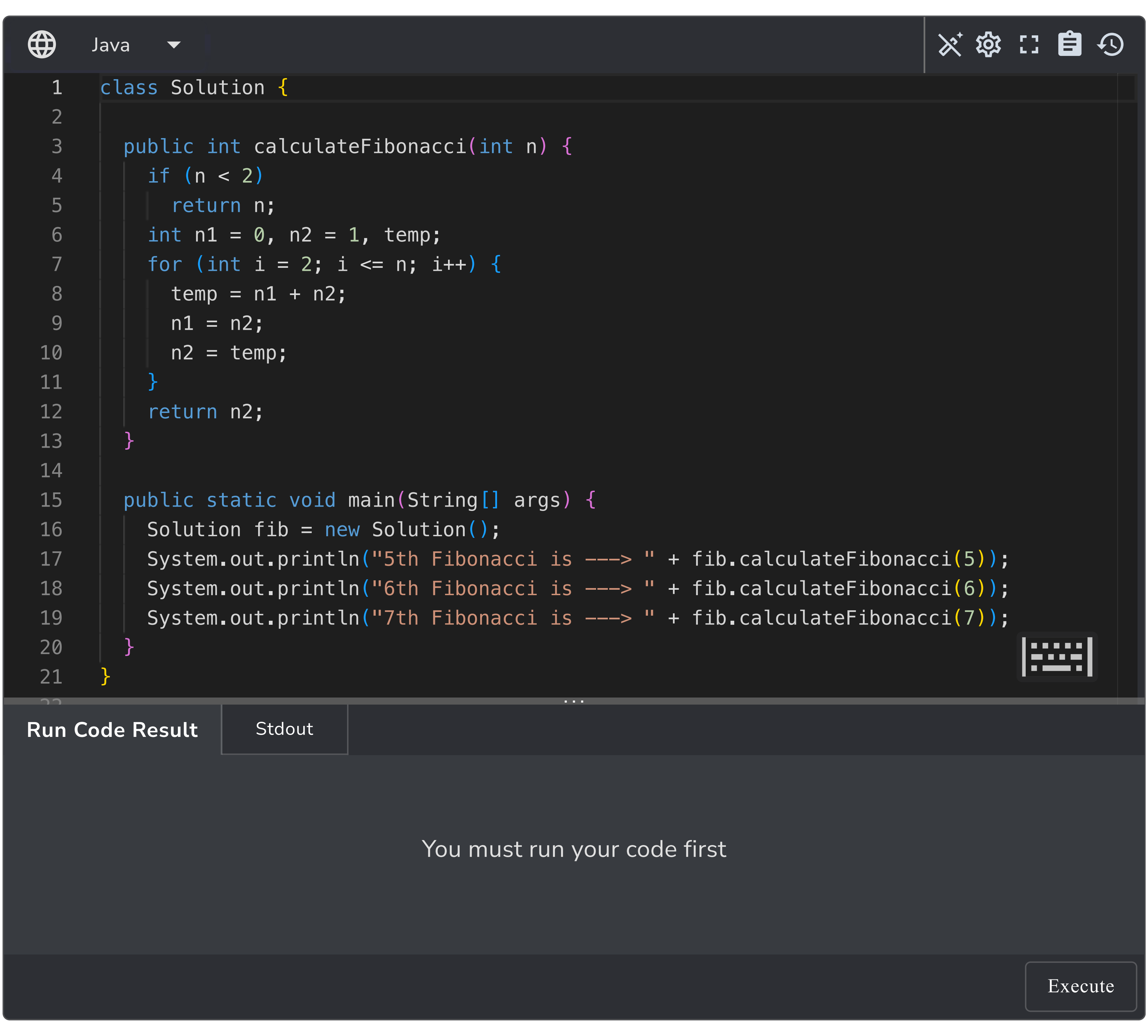
```java
class Solution {

  public int calculateFibonacci(int n) {
    if (n < 2)
      return n;

    int dp[] = new int[n + 1];
    dp[0] = 0;
    dp[1] = 1;
    for (int i = 2; i <= n; i++)
      dp[i] = dp[i - 1] + dp[i - 2];
    return dp[n];
  }

  public static void main(String[] args) {
    Solution fib = new Solution();
    System.out.println("5th Fibonacci is ---> " + fib.calculateFibonacci(5));
    System.out.println("6th Fibonacci is ---> " + fib.calculateFibonacci(6));
    System.out.println("7th Fibonacci is ---> " + fib.calculateFibonacci(7));
  }
}
```

You must run your code first

The above solution has time and space complexity of $O(n)$.

Memory optimization

We can optimize the space used in our previous solution. We don't need to store all the Fibonacci numbers up to 'n', as we only need two previous numbers to calculate the next Fibonacci number. We can use this fact to further improve our solution:



```java
class Solution {

  public int calculateFibonacci(int n) {
    if (n < 2)
      return n;
    int n1 = 0, n2 = 1, temp;
    for (int i = 2; i <= n; i++) {
      temp = n1 + n2;
      n1 = n2;
      n2 = temp;
    }
    return n2;
  }

  public static void main(String[] args) {
    Solution fib = new Solution();
    System.out.println("5th Fibonacci is ---> " + fib.calculateFibonacci(5));
    System.out.println("6th Fibonacci is ---> " + fib.calculateFibonacci(6));
    System.out.println("7th Fibonacci is ---> " + fib.calculateFibonacci(7));
  }
}
```

You must run your code first

The above solution has a time complexity of $O(n)$ but a constant space complexity $O(1)$.

← Previous
Fibonacci Numbers

Next →
Staircase

☐ Mark as Completed